LAPORAN HASIL PRAKTIKUM DAN TUGAS

Mobile Programming – Membangun Aplikasi Flutter dengan REST API Laravel

Sesi 11 (Integrasi REST API, Autentikasi Sanctum, MVVM + Riverpod)

Nama Mahasiswa	Ahmad Mofidur Robbiyal Aqla
NIM	2495114006
Kelas	5 - A
Judul Aplikasi	Perpustakaan (proyek perpus_app)
Teknologi	Flutter 3.47, Dart 3, Riverpod 3.5, Dio 5.11, SharedPreferences, Laravel 12 + Sanctum 4
Link Repository	https://github.com/RequimeOfficial/-Mobile-Programing-sesi-11

1. Tujuan

Praktikum ini membangun aplikasi Perpustakaan berbasis Flutter yang terhubung ke REST API Laravel (backend perpus-api). Aplikasi berjalan di Android dan Web dari satu codebase. Setelah praktikum, mahasiswa diharapkan mampu:

- menjelaskan pembagian tanggung jawab layer pada arsitektur MVVM (View, ViewModel, Repository, Service, Model);
- menghubungkan Flutter ke REST API memakai Dio, termasuk mengirim Bearer Token lewat interceptor;
- mengimplementasikan autentikasi Laravel Sanctum: register, login, logout, dan login otomatis;
- mengelola state aplikasi dengan Riverpod Notifier (loading, sukses, error);
- menangani error API: validasi 422 per field dan token kedaluwarsa (401);
- membangun UI responsif (HP dan Web) serta menguji kode dengan flutter analyze dan flutter test.

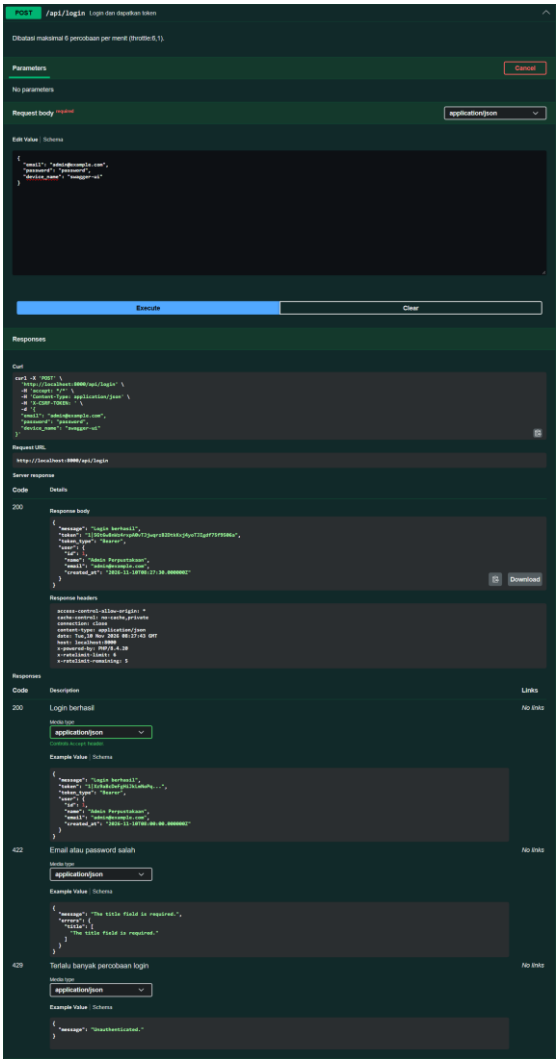
2. Persiapan

a. Backend perpus-api

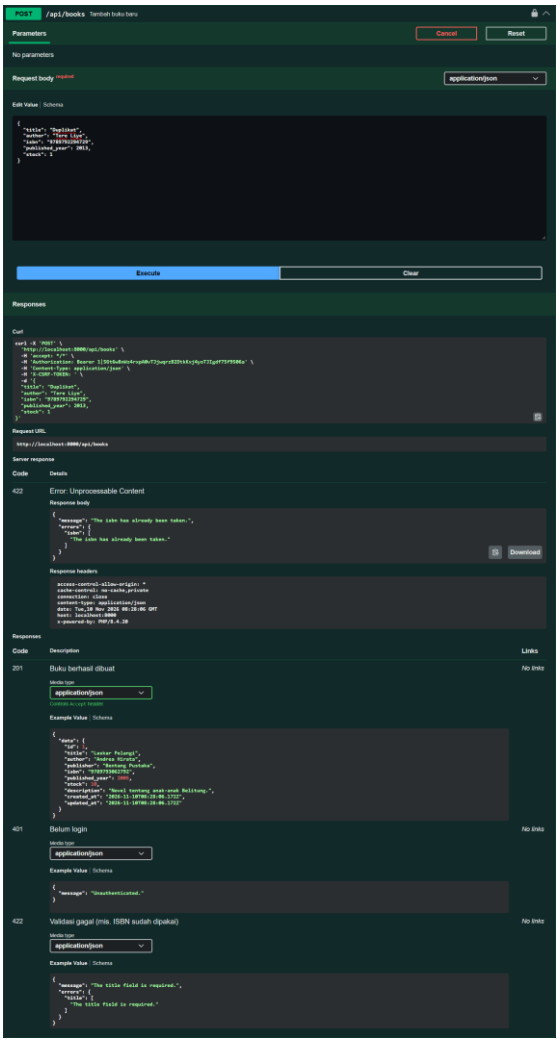
Backend yang dipakai adalah perpus-api (Laravel 12 + Sanctum + dokumentasi OpenAPI/Swagger) yang sebelumnya dibuat pada mata kuliah Pemrograman Web Lanjut. Database diisi data contoh lalu server dijalankan di port 8000. Akun demo: admin@example.com / password.

```
cd perpus-api
php artisan migrate:fresh --seed
php artisan serve        # -> http://localhost:8000
```

Latihan 1 (eksplorasi Swagger UI) dilakukan di http://localhost:8000/api/documentation: login lewat POST /api/login, menyalin token ke tombol Authorize, mencoba setiap endpoint, lalu mencoba POST /api/books dengan ISBN yang sudah ada sehingga server membalas 422.



Gambar 1. Swagger: POST /api/login (200) menghasilkan token



Gambar 2. Swagger: POST /api/books dengan ISBN duplikat (422)

b. Proyek Flutter

```
flutter create --platforms=android,web --project-name perpus_app Mobile-Programing-sesi-11
flutter pub add flutter_riverpod dio shared_preferences
```

Dependensi yang terpasang: flutter_riverpod ^3.5.0 (state management), dio ^5.11.1 (HTTP client) dan shared_preferences (penyimpanan token). Pada android/app/src/main/AndroidManifest.xml ditambahkan izin INTERNET dan android:usesCleartextTraffic="true" agar build release dapat mengakses backend lokal yang memakai http://. Base URL dipilih otomatis: localhost:8000 untuk web dan 10.0.2.2:8000 untuk emulator Android, atau diganti dengan --dart-define=API_BASE_URL untuk HP fisik.

3. Implementasi Praktikum (Arsitektur MVVM)

Kode disusun per layer. Data mengalir satu arah: View hanya membaca state dan memanggil method ViewModel; View tidak pernah memanggil Dio atau Repository secara langsung.

Layer	File	Tanggung jawab
Core	api_constants, dio_client, api_exception, app_router, app_theme	Base URL & endpoint, Dio + interceptor token/401, terjemahan error API, named route + navigatorKey, tema
Service	storage_service, api_service	Simpan/hapus token (dengan cache memori), wrapper GET/POST/PUT/DELETE
Model	user_model, book_model	DTO User dan Buku, BookPage membaca meta paginasi Laravel
Repository	auth_repository, book_repository	Register, login, pulihkan sesi, logout; CRUD buku + paginasi
ViewModel	providers, auth_vm, book_vm, profile_vm	Graf provider Riverpod, AuthState, BookState, profil
View	splash, login, register, books, book_form_dialog, book_detail, profile	Tampilan dan navigasi
Widget	responsive_layout, auth_scaffold, book_card, stock_style	Komponen yang dipakai ulang

a. Interceptor Dio dan ApiException

Interceptor menambahkan header Authorization: Bearer <token> pada setiap request. Jika server membalas 401 untuk request yang memakai token, token lokal dihapus dan pengguna diarahkan ke halaman login lewat navigatorKey global. Semua DioException diubah menjadi ApiException di ApiService sehingga ViewModel cukup menangkap satu jenis exception. ApiException juga menyimpan error per field dari validasi 422.

lib/core/network/dio_client.dart

```
onRequest: (options, handler) async {
  final token = await storage.getToken();
  if (token != null) options.headers['Authorization'] = 'Bearer $token';
  handler.next(options);
},
onError: (err, handler) async {
  final hadToken = err.requestOptions.headers.containsKey('Authorization');
  if (err.response?.statusCode == 401 && hadToken) {
    await storage.clearToken();
    if (err.requestOptions.path != ApiConstants.logout) onUnauthorized?.call();
  }
  handler.next(err);
},
```

b. Autentikasi Sanctum dan login otomatis

AuthRepository mengirim POST /api/login (dengan device_name), menyimpan token ke SharedPreferences, lalu mengembalikan UserModel. Saat aplikasi dibuka, SplashView memanggil restoreSession() yang memeriksa token tersimpan ke GET /api/me; jika masih

valid pengguna langsung masuk ke Daftar Buku. logout() memakai finally sehingga token lokal selalu terhapus walaupun server tidak dapat dihubungi.

c. State daftar buku (BookNotifier)

BookNotifier menyimpan daftar buku, status loading, halaman, dan total. load() memuat ulang dari halaman 1, loadMore() menambahkan halaman berikutnya (infinite scroll), sedangkan create/update mengembalikan ApiException? sehingga error ditampilkan di dialog form tanpa menghapus daftar. delete() tetap menghapus buku dari daftar lokal bila server membalas 404.

4. Latihan dan Tugas

a. Latihan 2 – Tracing alur data Tambah Buku (termasuk 422)

Alur berikut ditelusuri dari kode, dengan cabang ketika backend membalas 422 karena ISBN duplikat:

No	Dari → Ke	Aksi
1	User → BooksView	Menekan FAB "Tambah"
2	BooksView → BookFormDialog	openBookForm() → showDialog(BookFormDialog)
3	User → BookFormDialog	Mengisi form lalu menekan "Tambah"; _formKey.validate() (validasi lokal)
4	BookFormDialog → BookNotifier	setState(_saving = true); onSubmit(payload) → create(payload)
5	BookNotifier → BookRepository	createBook(payload)
6	BookRepository → ApiService/Dio	post('/books', payload); interceptor menambah Bearer token
7	Dio → Laravel	HTTP POST /api/books
8a	Laravel → … → BookNotifier	201 Created { data } → BookModel; state = [book, ...books], total + 1; return null
9a	BookFormDialog → BooksView	Navigator.pop(context, true) → snackbar "Buku berhasil ditambahkan"
8b	Laravel → ApiService	422 { errors: { isbn: [...] } } → throw ApiException.fromDio (fieldErrors[isbn])
9b	BookNotifier → BookFormDialog	return ApiException (state daftar buku tidak diubah)
10b	BookFormDialog → User	setState(_saving = false, _fieldErrors); dialog tetap terbuka, error di bawah field ISBN

b. Latihan 3 – Modifikasi tampilan stok

Fungsi warna dan label stok dibuat di lib/widgets/stock_style.dart agar dapat dipakai bersama oleh BookCard dan halaman detail. Stok 0 berwarna merah dengan label "Stok: 0 (Habis)", stok 1–5 oranye, dan stok lebih dari 5 hijau.

lib/widgets/stock_style.dart dan lib/widgets/book_card.dart

```
Color stockColor(int stock) {
  if (stock == 0) return Colors.red.shade700;
  if (stock <= 5) return Colors.orange.shade800;
  return Colors.green.shade700;
}

String stockLabel(int stock) => stock == 0 ? 'Stok: 0 (Habis)' : 'Stok: $stock';

// BookCard
subtitle: Text.rich(TextSpan(children: [
  TextSpan(text: '${book.author} · ${book.publishedYear} · '),
  TextSpan(text: stockLabel(book.stock),
      style: TextStyle(color: stockColor(book.stock), fontWeight: FontWeight.bold)),
])),
```

c. Tugas A – Halaman Detail Buku

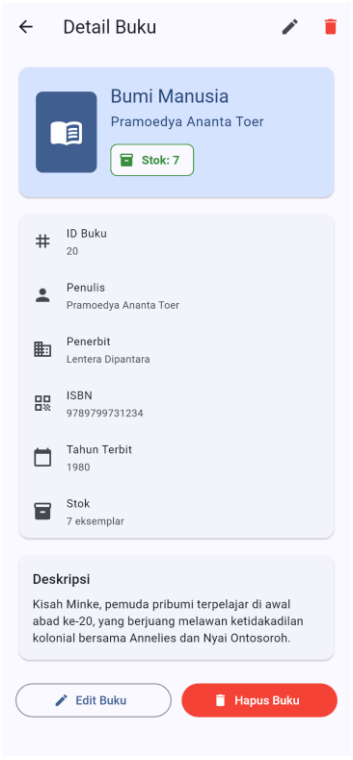
Dibuat BookDetailView (ConsumerWidget) di lib/views/books/book_detail_view.dart yang menampilkan semua field buku: judul, penulis, penerbit, ISBN, tahun terbit, stok (berwarna), ID, dan deskripsi, dengan tombol Edit dan Hapus. Route AppRoutes.bookDetail = '/books/detail' didaftarkan di main.dart dan BookCard kini membuka halaman ini dengan Navigator.pushNamed(..., arguments: book) menggantikan dialog detail. Halaman detail membaca versi terbaru buku dari bookProvider berdasarkan id, sehingga setelah Edit tampilan langsung berubah. Setelah Hapus berhasil, Navigator.pop mengembalikan pengguna ke daftar.

lib/views/books/book_detail_view.dart

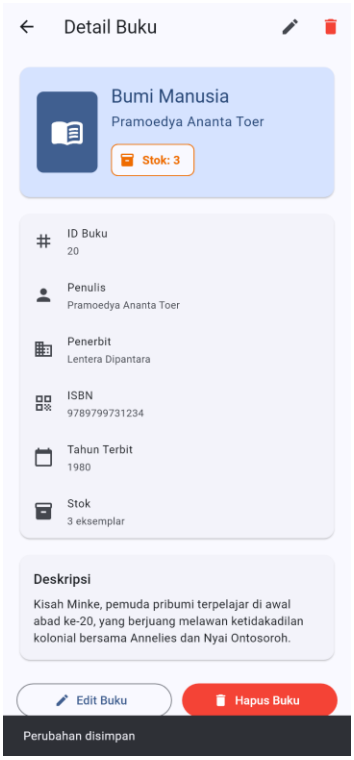
```
final current = ref.watch(bookProvider.select(
  (s) => s.books.where((b) => b.id == book.id).firstOrNull,
));
final b = current ?? book;

Future<void> delete() async {
  final deleted = await confirmDeleteBook(context, ref, b); // ->
bookProvider.notifier.delete
  if (deleted && context.mounted) Navigator.pop(context);
}
// Edit: openBookForm(context, ref, b) -> bookProvider.notifier.update(b.id, payload)
```

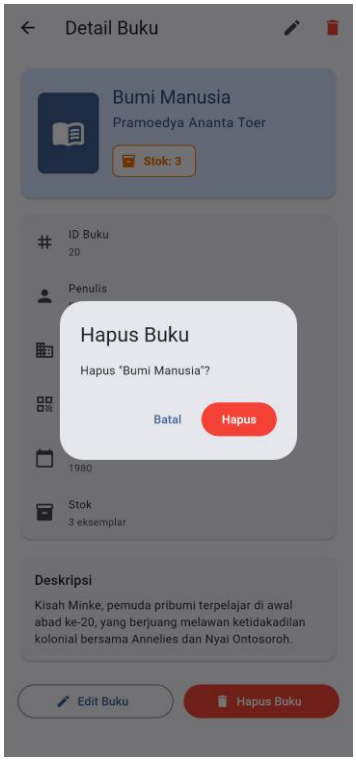
Agar tidak ada kode duplikat, dialog tambah/edit dan konfirmasi hapus dipindahkan ke lib/views/books/book_actions.dart (openBookForm, confirmDeleteBook, showToast) dan dipakai oleh BooksView maupun BookDetailView.



Gambar 3. Halaman Detail Buku



Gambar 4. Detail setelah stok diedit menjadi 3



Gambar 5. Konfirmasi hapus dari detail

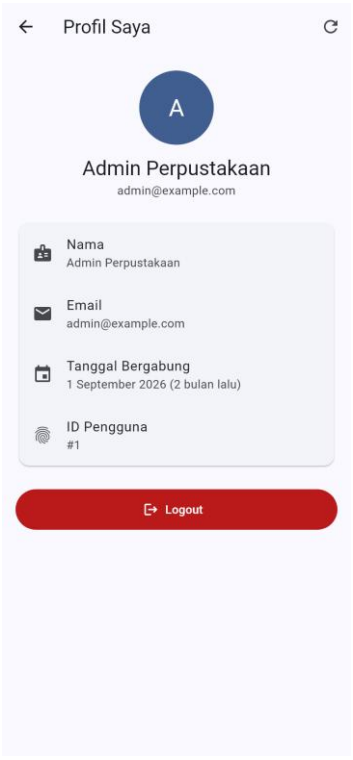
d. Tugas B – Halaman Profil User

UserModel ditambah field createdAt yang dibaca dari created_at (ISO 8601) lalu diubah ke waktu lokal. AuthRepository mendapat method getProfile() (GET /api/me) yang juga dipakai ulang oleh restoreSession(). Di layer ViewModel dibuat profileProvider (FutureProvider.autoDispose) sehingga ProfileView cukup membaca AsyncValue (loading / data / error) dan tidak memanggil API langsung. Halaman profil menampilkan nama, email, tanggal bergabung (format "1 September 2026 (2 bulan lalu)"), ID pengguna, dan tombol Logout. Tujuan Profil ditambahkan di NavigationRail (desktop) dan tombol profil di AppBar (HP).

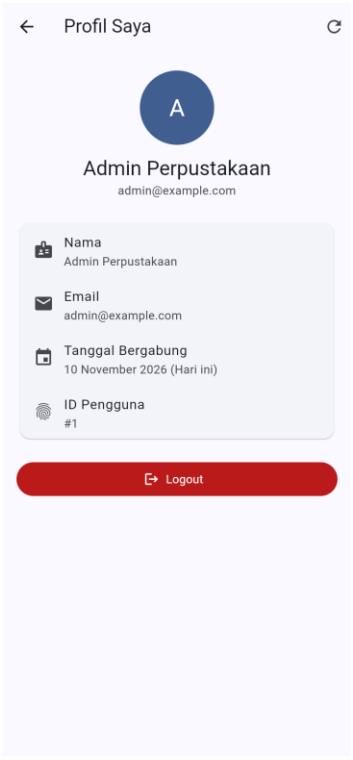
lib/viewmodels/profile_vm.dart dan lib/repositories/auth_repository.dart

```
final profileProvider = FutureProvider.autoDispose<UserModel>(
  (ref) => ref.read(authRepositoryProvider).getProfile(),
);

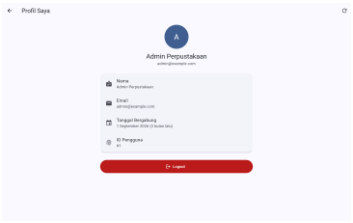
Future<UserModel> getProfile() async {
  final res = await _api.get(ApiConstants.me);
  return UserModel.fromJson(res.data['data'] as Map<String, dynamic>);
}
```



Gambar 6. Profil (tampilan HP)



Gambar 7. Profil dari GET /api/me backend asli



Gambar 8. Profil (tampilan desktop)

e. Tugas C – Pencarian otomatis (debounce)

Pencarian sebelumnya baru berjalan setelah Enter. Sekarang TextField pencarian memakai onChanged yang menjalankan Timer 500 ms; setiap huruf baru membatalkan Timer sebelumnya, sehingga request hanya dikirim setelah pengguna berhenti mengetik. Enter dan tombol hapus tetap mencari langsung, dan Timer dibatalkan di dispose().

lib/views/books/books_view.dart

```
Timer? _debounce;

void _onSearchChanged(String value) {
  _debounce?.cancel();
  _debounce = Timer(const Duration(milliseconds: 500), () {
    if (value.trim() == ref.read(bookProvider).search) return; // tidak berubah
    _vm.load(search: value);
  });
}

void _searchNow(String value) { _debounce?.cancel(); _vm.load(search: value); }
// TextField: onChanged: _onSearchChanged, onSubmitted: _searchNow
```

Pengujian di browser terhadap backend asli: mengetik "dolore" (6 huruf) hanya menghasilkan satu request GET /api/books?search=dolore&page=1&per_page=15 (lihat screenshots/api_asli/network_log.txt).

f. Pertanyaan analisis: mengapa BookNotifier memakai _requestId?

Setiap pemanggilan load() menaikkan _requestId dan menyimpan nilainya di variabel lokal id. Ketika respons datang, kode memeriksa if (id != _requestId) return; sehingga hanya respons dari request terakhir yang boleh mengubah state. Tanpa _requestId, bila pengguna mencari "a" lalu cepat mengganti menjadi "ab", kedua request berjalan bersamaan. Jika respons "ab" datang lebih dulu lalu respons "a" datang belakangan, daftar akan ditimpa hasil "a" padahal kolom pencarian berisi "ab" (race condition). Data yang tampil tidak sesuai kata kunci, dan paginasi loadMore() ikut melanjutkan dari hasil yang salah. Dengan _requestId respons lama dikenali sebagai usang dan diabaikan; reset() saat logout juga menaikkan _requestId agar respons yang masih berjalan tidak masuk ke sesi pengguna berikutnya. Debounce mengurangi jumlah request, tetapi tidak menggantikan _requestId karena jeda mengetik lebih dari 500 ms tetap dapat menghasilkan dua request yang responsnya datang tidak berurutan. Kasus ini diuji pada test/tugas_test.dart.

5. Hasil Pengujian

a. flutter analyze dan flutter test

Kode diperiksa dengan flutter analyze (tanpa issue) dan diuji dengan 18 test: 7 test bawaan modul (test/widget_test.dart) dan 11 test baru untuk Latihan 3 serta Tugas A, B, C (test/tugas_test.dart). Test memakai repository palsu sehingga tidak membutuhkan server.

```
Terminal - perpus_app
> flutter analyze
Analyzing Mobile-Programing-sesi-11...
No issues found! (ran in 60.5s)

> flutter test
widget_test.dart  Model & parsing respons backend: BookPage.fromJson membaca data + meta paginasi
widget_test.dart  Model & parsing respons backend: ApiException membaca error validasi 422
widget_test.dart  Model & parsing respons backend: ApiException memberi pesan jelas saat server mati
widget_test.dart  BookNotifier: load + loadMore menggabungkan halaman
widget_test.dart  BookNotifier: create gagal tidak menghapus daftar & mengembalikan error field
widget_test.dart  BookNotifier: delete menghapus buku dari daftar
widget_test.dart  Form buku: validasi lokal & error ISBN dari backend tampil
tugas_test.dart   Latihan 3: warna & label stok sesuai aturan
tugas_test.dart   Latihan 3: BookCard menampilkan stok habis berwarna merah
tugas_test.dart   Tugas A: tap kartu membuka detail berisi semua field
tugas_test.dart   Tugas A: edit dari detail langsung memperbarui tampilan
tugas_test.dart   Tugas A: hapus dari detail kembali ke daftar & buku hilang
tugas_test.dart   Tugas B: UserModel membaca created_at ISO 8601
tugas_test.dart   Tugas B: tombol profil (HP) membuka profil dari GET /api/me
tugas_test.dart   Tugas B: NavigationRail (desktop) punya tujuan Profil
tugas_test.dart   Tugas B: logout dari profil kembali ke halaman login
tugas_test.dart   Tugas C: mengetik cepat hanya mengirim 1 request setelah 500 ms
tugas_test.dart   Tugas C: _requestId: respons pencarian lama yang datang belakangan diabaikan
00:09 +18: All tests passed!

> flutter build apk --debug
Running Gradle task 'assembleDebug'...                          682.8s
√ Built build\app\outputs\flutter-apk\app-debug.apk
```

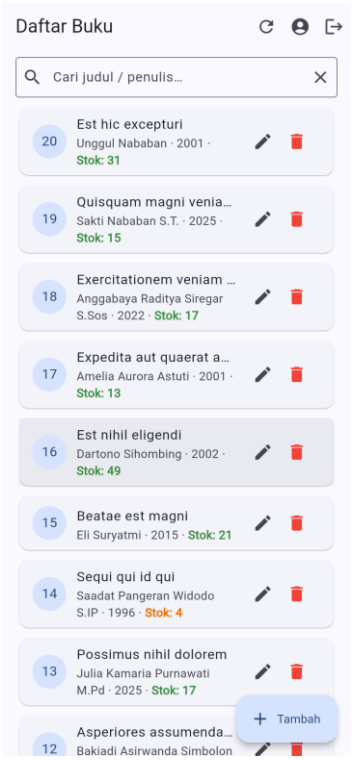
Gambar 9. Hasil flutter analyze, flutter test, dan flutter build apk --debug

b. Checklist uji manual

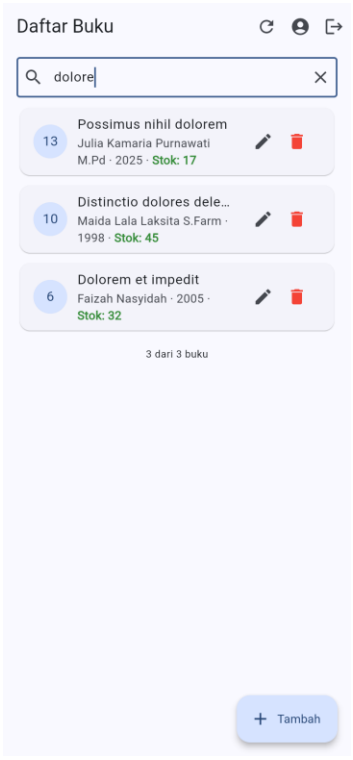
Aplikasi dijalankan sebagai Flutter Web (flutter build web) di Chrome dan terhubung ke backend perpus-api asli di localhost:8000. Seluruh skenario pada checklist modul berhasil.

No	Skenario	Hasil yang diharapkan	Status
1	Login dengan password salah	Pesan "Email atau password salah." di bawah field Email	Berhasil
2	Login admin@example.com / password	Masuk ke Daftar Buku	Berhasil
3	Scroll sampai bawah	Halaman berikutnya dimuat, footer "20 dari 20 buku"	Berhasil
4	Tambah buku dengan ISBN yang sudah ada	Dialog tetap terbuka, error di field ISBN	Berhasil
5	Tambah buku dengan ISBN baru	Buku muncul di atas daftar + snackbar	Berhasil
6	Edit stok buku	Stok berubah + snackbar "Perubahan disimpan"	Berhasil
7	Cari judul	Daftar hanya berisi buku yang cocok	Berhasil
8	Hapus buku	Buku hilang dari daftar + snackbar	Berhasil
9	Reload browser	Langsung masuk Daftar Buku tanpa login (GET /api/me)	Berhasil
10	Register dengan email terdaftar	Pesan error di bawah field Email	Berhasil
11	Logout	Kembali ke halaman login	Berhasil
12	Lebar < 768 px dan ≥ 768 px	Tampilan HP (FAB) dan desktop (NavigationRail)	Berhasil

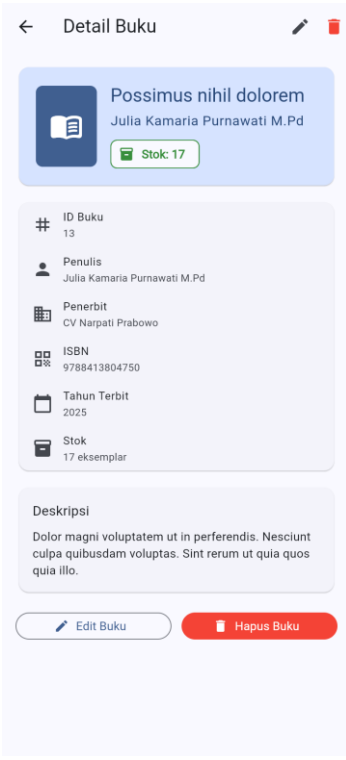
c. Tangkapan layar dengan backend asli



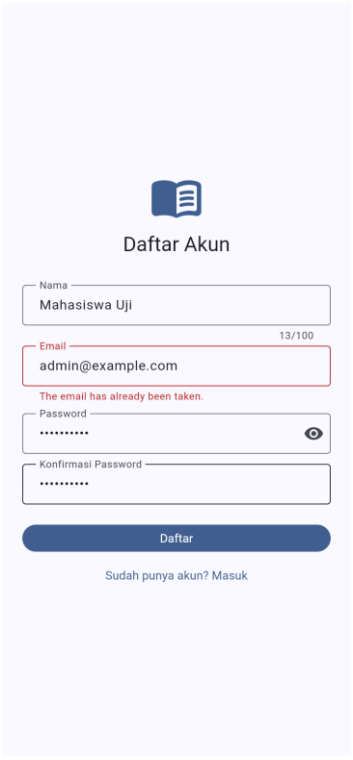
Gambar 10. Daftar buku dari GET /api/books



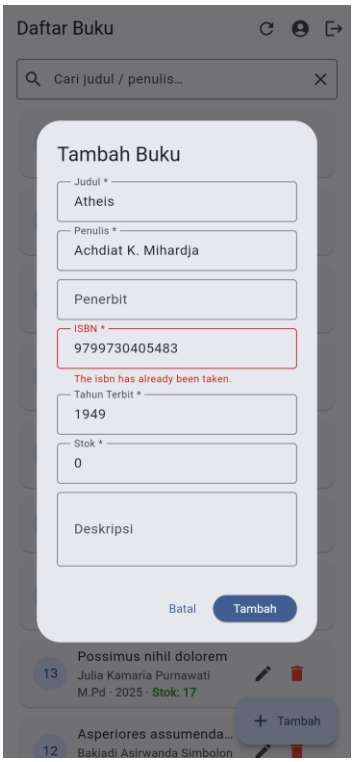
Gambar 11. Pencarian "dolore" (1 request)



Gambar 12. Detail buku dari data asli



Gambar 13. Register email duplikat (422)



Gambar 14. Tambah buku ISBN duplikat (422)

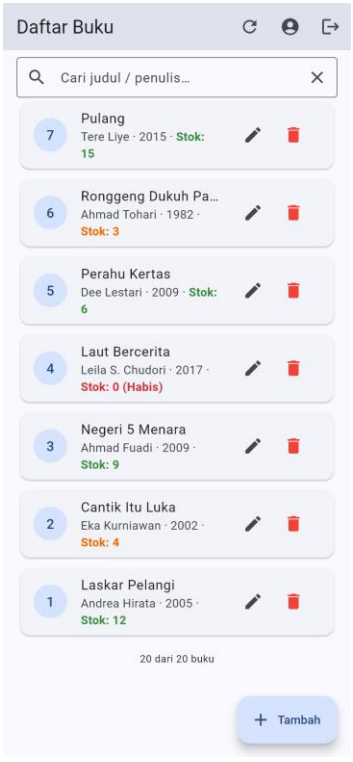


Gambar 15. Login gagal (422)

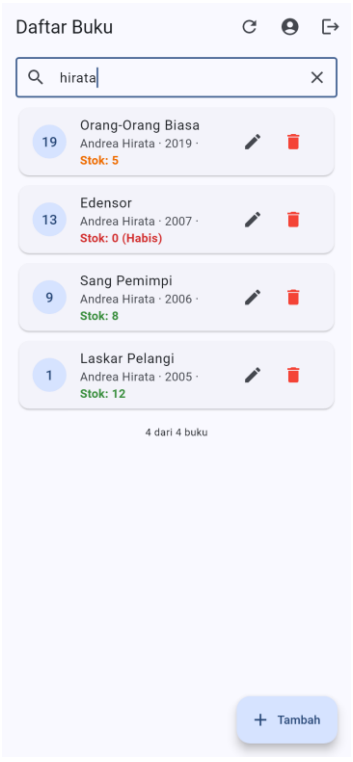
d. Alur aplikasi (tampilan HP)



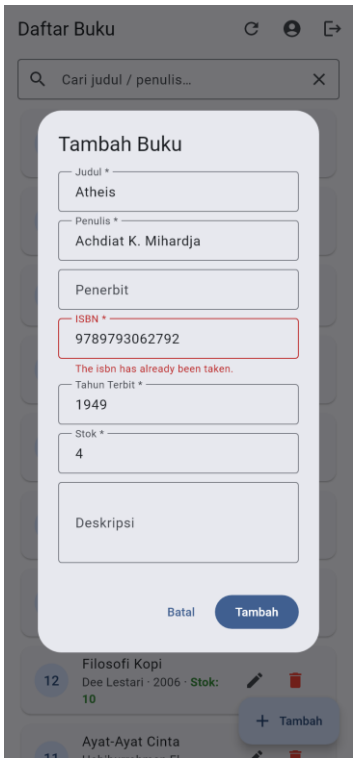
Gambar 16. Daftar buku + warna stok



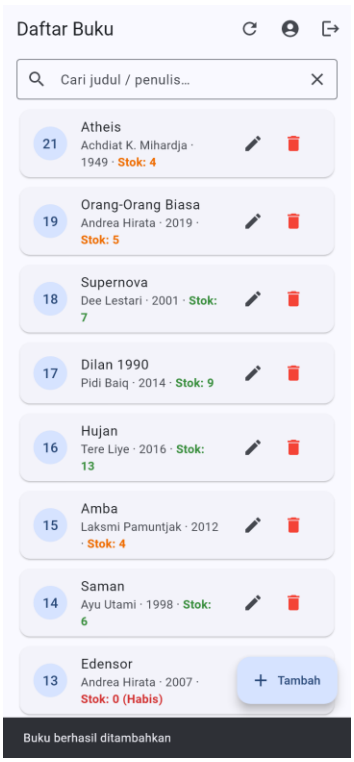
Gambar 17. Infinite scroll: 20 dari 20 buku



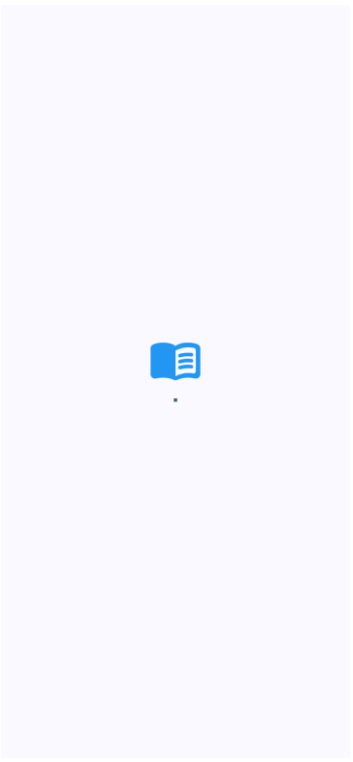
Gambar 18. Pencarian otomatis "hirata"



Gambar 19. Dialog tambah: error ISBN

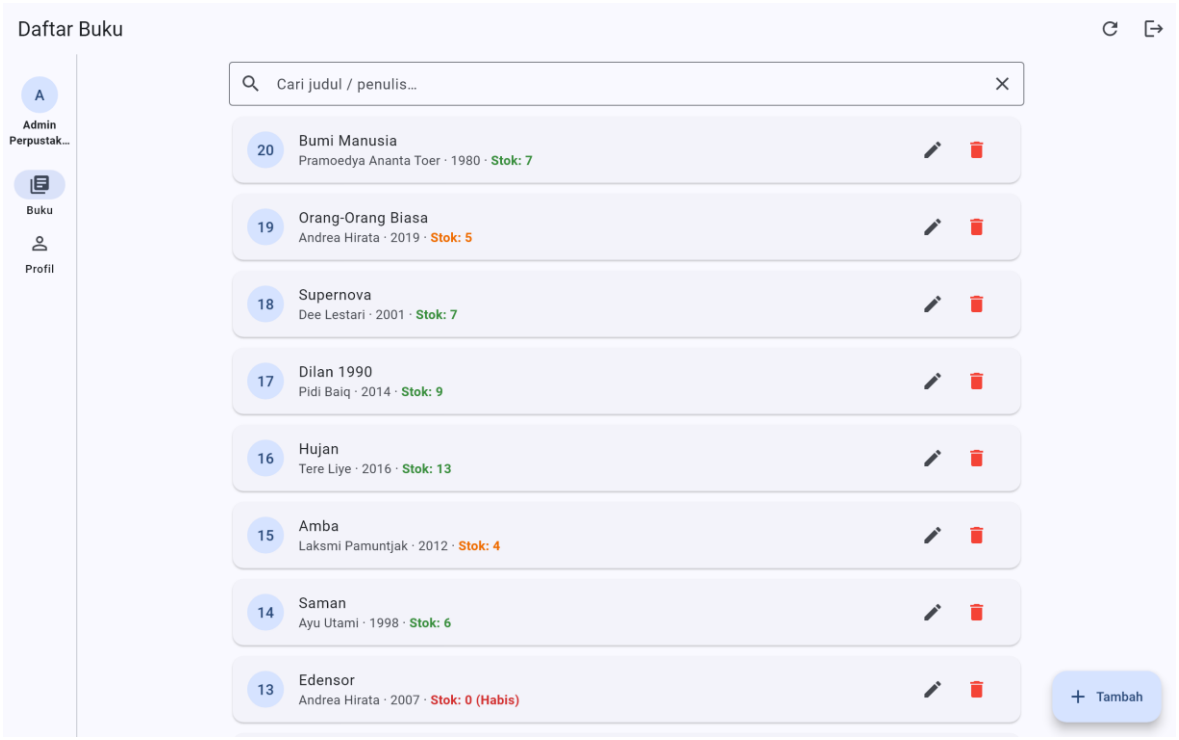


Gambar 20. Buku baru di urutan teratas

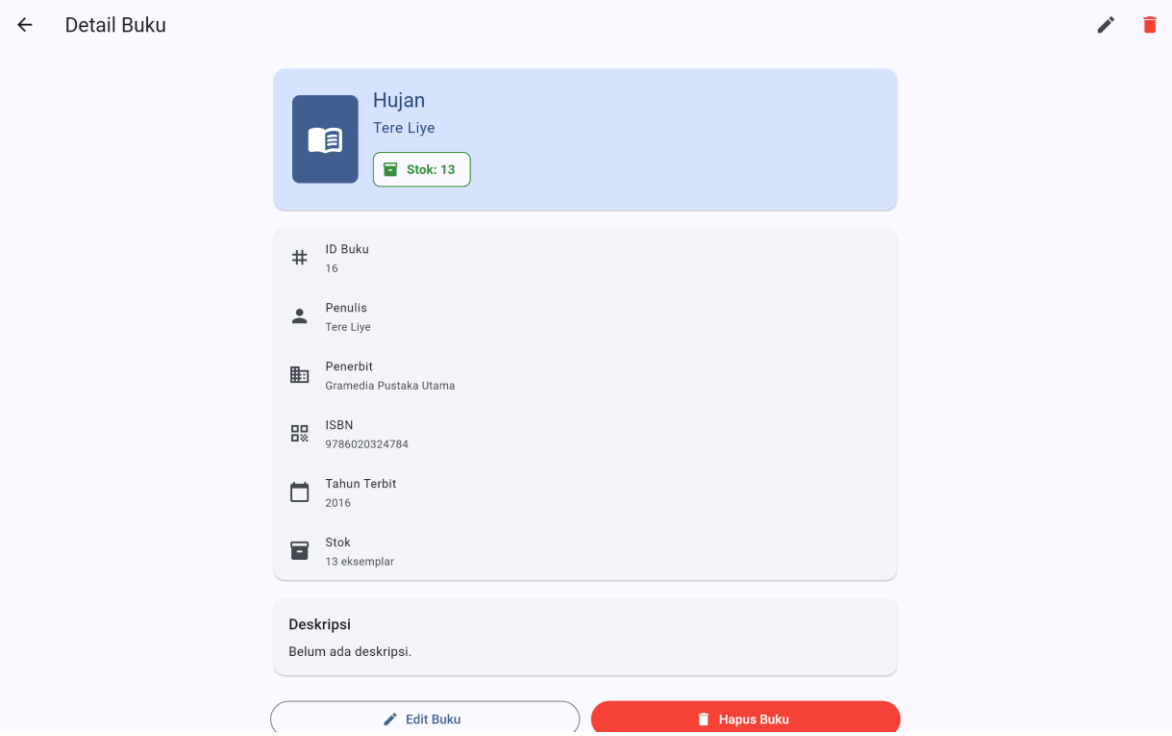


Gambar 21. SplashView memeriksa sesi

e. Tampilan web/desktop (≥ 768 px)



Gambar 22. Daftar buku dengan NavigationRail (Buku dan Profil)



Gambar 23. Halaman Detail Buku pada layar lebar

f. Video demonstrasi

Video demonstrasi berdurasi sekitar 1 menit 44 detik (video/demo_perpustakaan.mp4) memperlihatkan alur splash, login gagal dan berhasil, warna stok, infinite scroll, pencarian dengan debounce, halaman detail beserta edit dan hapus, tambah buku dengan ISBN duplikat lalu berhasil, halaman profil, dan logout. Video tersedia di repository: https://github.com/RequimeOfficial/-Mobile-Programing-sesi-11.

6. Kesimpulan

Aplikasi Perpustakaan berhasil dibangun dengan arsitektur MVVM: Service menangani HTTP dan penyimpanan token, Repository mengubah JSON menjadi model, ViewModel (Riverpod Notifier) mengelola state, dan View hanya membaca state serta memanggil method ViewModel. Autentikasi Sanctum (register, login, login otomatis, logout) dan CRUD buku dengan paginasi berjalan baik terhadap backend perpus-api, termasuk penanganan error 422 per field dan 401. Tugas A (Detail Buku), Tugas B (Profil User dari GET /api/me), Tugas C (pencarian dengan debounce), dan Latihan 3 (warna stok) selesai, dengan flutter analyze tanpa issue dan 18 test lulus.

Kendala selama pengerjaan

- **Detail tidak ikut berubah setelah diedit.** Data buku awalnya hanya diambil dari arguments route. Masalah diatasi dengan membaca versi terbaru dari bookProvider (ref.watch + select berdasarkan id).
- **Kode dialog terduplikasi.** Dialog tambah/edit/hapus dibutuhkan di BooksView dan BookDetailView, begitu juga logout di BooksView dan ProfileView, sehingga dipindah ke book_actions.dart dan session_actions.dart.

- **Debounce dan respons lama.** Debounce mengurangi request saat mengetik, tetapi respons pencarian lama masih mungkin datang terlambat; hal ini ditangani oleh _requestId yang sudah ada di BookNotifier.
- **Base URL berbeda per platform.** Di emulator Android localhost menunjuk ke emulator itu sendiri, sehingga dipakai 10.0.2.2, sedangkan Web memakai localhost dan HP fisik memakai --dart-define.